Git Introduction Course

Romain De Groote

February 23, 2026

Contents

1 Introduction **2**
1.1 Why Git? . . . 2
1.2 What is Git? . . . 2

2 Starting with Git **3**
2.1 Installing Git . . . 3
2.2 Setting up Git . . . 3
2.3 Some tools to use Git with and how to use them . . . 3

3 How to work with git **6**
3.1 Main/Master branch . . . 6
3.2 Branching . . . 6
3.3 Preparing a commit . . . 7
3.4 Getting the updated version . . . 7
3.5 Accepting a Pull Request . . . 8

4 Conclusion **9**
4.1 To go further… . . . 9

A Appendix **10**
A.1 CheatSheets . . . 10
A.2 Markdown . . . 12

1 Introduction

Welcome to this Git introduction course! In this course we will cover the basic of git, a popular version control system used by developers worldwide. By the end of this course, you will be able to use git to manage your code and collaborate with others effectively.

1.1 Why Git?

For any person who has work at least one time on a group project, one of the main issue was to share the project between the different actors. There is several options to do so, each with their pros and cons:

- USB stick/SD card
  - Pros: It is sure the files are exactly the same for everyone, easiest to put in place
  - Cons: Only one person at a time can have it, no history if the file is lost
- Drive (Onedrive, Google drive, …)
  - Pros: Everyone can access the files at anytime, quick to put in place
  - Cons: require external account, limited to some apps, no history if the file is lost
- Git
  - Pros: Everyone can work on its 'own' version of the project, easy merge, full history of commit
  - Cons: Needs to create a repository, (only non binary files)

With this quick comparison, we can see that the Drive and the Git repository have similar pros and cons. The main difference being that Git requires to be setup manually where a drive only need to be share and voilà, it is done. However, looking beneath this difference reveals a much more dire one: one between two (2) paradigmes:

- Onedrive and Google drive are proprietary software, meaning that the code, and most importantly your files, are hosted on private servers. You have to trust the company to not use your files against you (for ads for example) and to not loose them.
- Git is 'open source' and do not need any subscription and can be host on your own server. This means that you have the full control of your files and that you can share them with who you want, how you want.

For you who will work in Computer Science or even in any other field of science, you have to know that Git is widely used and that soon or later you will have to face it. It's openess allows to share easily open source project such as:

- Mistral AI
- Python
- Linux Kernel
- Doom PDF[1]

1.2 What is Git?

Git is a distributed version control system created by Linus Torvalds in 2005 to help manage the development of the Linux kernel, now maintained by Junio Hamano. It allows multiple developers to work on the same codebase simultaneously, track changes, revert to previous versions, and collaborate effectively. It is designed to be fast, efficient, and scalable, making it suitable for projects of all sizes.

[1] Yes, *Doom*, the famous FPS has been ported to be playable in a PDF file.

2 Starting with Git

2.1 Installing Git

2.1.1 On Windows

To install git, just go to git-scm/windows and download the installer. Then, just follow the instructions.

2.1.2 On Linux

To install git on Linux, you can use your package manager. See git-scm/linux for the exact command for your distribution.

2.1.3 On MacOS

To install git on MacOS, you can use Homebrew (if you don't have it, go to the website and follow the instructions). Then, just run the command:

```
brew install git
```

As always, you can also see git-scm/macos for more options.

2.2 Setting up Git

After installing git, you need to set up your user name and email. This is important as it will be used to identify you in the commit history. To do so, just run the following commands in your terminal:

```
git config --global user.name "Your Name"
git config --global user.email "Your email"
```

For all installation and configuration, always use a personnal email to avoid issue when you'll leave your current school or job.

2.3 Some tools to use Git with and how to use them

As said earlier, the original way of using Git is going through the terminal. Hopefully for you, nowadays there is several great user interface to use it, so don't hesitate to use them![2]

2.3.1 GitHub

One of the most famous Git repository hosting service is GitHub[3]. To use it, you can either use the web interface or use some UI such as GitHub Desktop to manage your repositories both locally and remotely.

1. To use GitHub, you need to create an account first, for this go to GitHub signup.
2. Then, you can create a new repository by clicking on the "Create New" button on the main page.
3. You then arrive on this page where you can fill the information about your repository[4].
    - Name: The name of your repository
    - Description: A short description of your repository
    - Visibility: Choose between Public (anyone can see your repository) or Private (only you and people you share it with can see it)
    - Initialize this repository with a README: If you check this box, a README file will be created for you automatically.

[2]Knowing the Git Bash command is still usefull as not every project manager uses a comprehensive UI (see Overleaf for this).
[3]To be noted that GitHub is owned by Microsoft. If you want to avoid it, you can use GitLab or host your own Git server.
[4]See a repository as a folder containing your project files. Its name should be relevant to the project itself and must be unique on your account/main folder.

- .gitignore: A file that tells Git which files to ignore (not track). You can choose a template based on the type of project you are working on.
- License: A file that tells others what they can and cannot do with your code. You can choose a license based on your needs.

4. Finally, click on the "Create repository" button to create your repository.

2.3.1.1 GitHub Desktop Once you have created your repository, you can use GitHub Desktop to manage it. To do so, just download and install GitHub Desktop from here. Then, open GitHub Desktop and sign in with your GitHub account. You can then clone your repository to your local machine by clicking on the "File" menu and slecting "Clone repository…".

Then select your repository and choose the local path where you want to clone it.

Congratulations, you have now cloned your repository to your local machine! You can now use GitHub Desktop to manage your repository both locally and remotely.

2.3.2 Gitlab

GitLab is another popular Git repository hosting service. It is open source and can be self-hosted. You can use the web interface or use some UI such as GitKraken to manage your repositories both locally and remotely.

1. To use GitLab, you need to create an account first, for this go to GitLab signup.
2. Then, you can create a new repository by clicking on the "Project" tab on the main page.
3. Then click on the "New project" button to create a new repository.
4. You can then choose to create a blank project, import a project from GitHub or GitLab, or create a project from a template.
5. if you select a blank project, you will arrive on this page where you can fill the information about your repository:
   - Name
   - Project group or namespace
   - Project slug[5]
   - Deployement target
   - Visibility level
   - Initialize repository with a README
   - Enable SAST
   - Enable Secret Detection
6. Finally, click on the "Create project" button to create your repository.

[5]See a repository as a folder containing your project files. Its name should be relevant to the project itself and must be unique on your account/main folder.

2.3.3 Git Bash

To use Git bash, just open your file explorer, search `git bash` and open it. A bash console will appear, allowing you to navigate through your files and use git command along bash ones.

1. Navigate where you want to clone your git repository or to create it.[6]

   ```
   cd <path/to/location>
   ```

2. Once there, you can either clone a repository with:

   ```
   git clone <url> <foldername>
   ```

   or create a repository using

   ```
   git init
   ```

You can now open your git folder in your favorite IDE.

2.3.4 Setting up a git server

You may want to be totally independent of any online platform for privacy, security and controll over your data and your git repository. For this it is possible to create your own git server hosted on your machine or on your own server. As this is a more advanced part, we won't cover it in this course.

2.3.5 Overleaf

Overleaf is popular online LaTeX editor that also provides Git integration. You can use the web interface to edit your LaTeX documents and use Git to manage your project versioning.[7]

1. First open your Overleaf project
2. Open the integration menu
3. Select the Git option
4. Copy the command
5. Follow the git bash instructions to go to the right folder and clone your project repository

Don't forget reconnect to Internet to push your changes to the remote repository and see them on Overleaf. (The push and pull are done automatically by Overleaf when you are connected to Internet, so you don't have to do anything special)

[6] See a repository as a folder containing your project files. Its name should be relevant to the project itself and must be unique on your account/main folder.

[7] Unfortunately, Overleaf Git integration is only available for premium users only, however if the owner of the project owns it, you can use it too.

3 How to work with git

3.1 Main/Master branch

When you create your git project, you start in the main (or master) branch. From there, you can create some new branches, allowing you to work on a copy of the file *without* modifying the original one. Some pros of this:

- What you have done is useless or depreceted? you can just delete the branch and come back to your master branch
- Several people can start working with the same basis on different part of the project
- You can test new features or debug without breaking the main code. When you are satisfied with your changes, you can merge them back to the main branch, if not just delete the branch.
- You can work on several features at the same time without mixing them up.

However, branches can lead to some issues if not used properly:

- If several people work on the same file, merging can become a pain. Git will try to merge the changes automatically, but if it can't, you will have to do it manually.
- If you work on a branch for a long time without merging it back to the main branch, you may have to deal with a lot of conflicts when you finally try to merge it back.
- If you create too many branches, it can become hard to manage them all.
- If you delete a branch, you may lose some important changes if you haven't merged them back to the main branch.
- If you forget to switch back to the main branch before starting a new feature, you may end up working on the wrong branch.
- If you don't name your branches properly, it can become hard to know what each branch is for.
- While working with a team, it is important to *communicate* about the branches each member is working on, when they plan to merge them back, and, most importantly, to organize to know who is working on what to avoid conflicts.
- **Be aware to not merge in the wrong branch or way. Like merging the *main* branch into a feature branch by mistake.**

With all this in mind, Git might seem a bit complicated and hard to use it at first, but with practice and discipline, it becomes a powerful tool to manage your projects and collaborate with others.

In fact, the main branch should be consider as the stable version of your project, the only commit allowed should be merge from other branches that have been tested and validated. In fact, it is a good practice to protect the main branch from direct commits, forcing everyone to work on branches and merge them back only when they are ready.

3.2 Branching

To create a new branch, you can use the command:

```
git checkout -b <branch_name>
```

or

```
git switch -c <branch_name>
```

the `-b` or `-c` flag tells git to create the branch if it doesn't exist. To switch between branches, you can use the command:

```
git checkout <branch_name>
```

or

```
git switch <branch_name>
```

When you have finished working on your branch and want to merge it back to the parent branch, you can use the command:

```
git checkout <parent_branch>
git merge <branch_name>
```

Be aware to be on the parent branch before merging, `merge` command will merge the specified branch into the current branch, not the other way around.

when you have definitively finished with a branch, you can delete it using the command:

```
git branch -d <branch_name>
```

be careful when deleting branches, as you may lose important changes if you haven't merged them back to the main branch, **deleted branches are forever lost**.

3.3 Preparing a commit

While working with git, you will at first work locally on your files. Once you have made the changes you want, you need to prepare them to be committed. For this, several commands will be useful:

1. First, you can add the files you want to commit to the staging area using the command:

   ```
   git add <file_name>
   ```

   or to add all the changes at once:

   ```
   git add .
   ```

2. You can then commit the changes using the command:

   ```
   git commit -m "commit message"
   ```

   The `-m` flag allows you to add a commit message directly from the command line. If you don't use it, git will open your default text editor to write the commit message. If you want to stage and commit all changes at once, you can use the command:

   ```
   git commit -am "commit message"
   ```

   The `-a` flag tells git to stage all changes before committing.

3. Finally, you can push the changes to the remote repository using the command:

   ```
   git push origin <branch_name>
   ```

   This will push the changes to the specified branch on the remote repository.

Pushing the changes will make them available to everyone who has access to the repository. However, if you are not proprietary of the repository, you may need to create a pull request to have your changes reviewed and merged by the repository owner. The process of creating a pull request varies depending on the platform you are using (GitHub, GitLab, Bitbucket, etc.), but generally involves going to the repository page and clicking on the "New Pull Request" button.

3.4 Getting the updated version

To get the updated version of a git repository, you can use the command

```
git pull origin <branch_name>
```

With this you'll get all the changes made by your teammates on the specified branch. If you are on the branch you want to update, you can simply use:

```
git pull
```

and if you want to update all branches, you can use:

```
git fetch --all
```

3.5 Accepting a Pull Request

When someone creates a pull request[8], you'll have to review and accept the pull request, you can follow these steps:

1. Go to the repository page on the platform you are using (GitHub, GitLab, Bitbucket, etc.).
2. Click on the "Pull Requests" tab to see the list of open pull requests.
3. Click on the pull request you want to review.
4. Review the changes made in the pull request. You can see the files changed, the commit history, and any comments made by the author.
5. If you are satisfied with the changes, you can click on the "Merge Pull Request" button to merge the changes into the main branch.
6. If you have any comments or suggestions, you can leave them in the pull request for the author to address before merging.

[8] A pull request is a bit tricky term. It doesn't mean that you ask to get the new updates, but that you ask to push on a repository you don't own.

4 Conclusion

Congratulations, you can now use Git to manage your projects and collaborate with others! Don't hesitate to explore more advanced features of Git and the platform you are using to make the most out of it. (e.g., GitHub Actions, GitLab CI/CD, Git history manipulation, etc.).

Git is a powerful open source tool that can greatly enhance your development workflow, so keep practicing and learning!

If you have any questions or need help, don't hesitate to ask!

To end this course, here are some cheat sheets to help you remember the most important commands and concepts.

4.1 To go further...

Here some quick tips to go further with Git:

- Use `.gitignore` files to exclude files and directories from being tracked by Git. This is useful for temporary files, build artifacts, and sensitive information.
- Learn about branching strategies such as Git Flow or GitHub Flow to manage your branches and releases more effectively.
- Explore Git hooks to automate tasks such as running tests or formatting code before committing changes.
- Look up for some extension or how your favorite IDE integrate git.
- Enhance your readme file with this following chapter.

A Appendix

A.1 CheatSheets

A.1.1 Bash

Here some basic bash commands to navigate and manipulate files and directories:

```
cd <path/to/folder>          # Change directory to the specified folder
ls                           # List files and directories in the current directory
cp <source> <destination>    # Copy a file or directory
mv <source> <destination>    # Move or rename a file or directory
rm <file>                    # Remove a file
mkdir <directory>            # Create a new directory
rmdir <directory>            # Remove an empty directory
touch <file>                 # Create a new empty file
```

A.1.2 Git

Here some basic git commands to manage your repository[9]:

A.1.2.1 Refering to a commit

- brach: `main`
- tag: `v1.0`
- commit ID: `a1b2c3d4`
- remote: `origin/main`
- current commit: `HEAD`
- x commits before HEAD: `HEAD~x` or `HEAD^...^` (e.g., `HEAD~2` or `HEAD^^` for two commits before HEAD)

A.1.2.2 Getting Started

```
## Getting started
git init                     # Initialize a new git repository
git clone <repository_url>   # Clone an existing repository
```

A.1.2.3 Commit

```
## Prepare to Commit
git add <file>               # Stage a file for commit
git add .                    # Stage all changes for commit
git add -p                   # Choose which parts of a file to stage
git mv <old> <new>           # Rename a file and stage the change
git rm <file>                # Remove a file and stage the change
git rm --cached <file>       # Unstage a file but keep it in the working directory
git reset <file>             # Unstage a file
git reset                    # Unstage all changes
git status                   # Show the status of the working directory and staging area

## Make Commits
git commit -m "message"      # Commit staged changes with a message
git commit                   # Commit staged changes with an editor to write the message
git commit -am "message"     # Stage and commit all changes with a message
```

[9] List taken from Git - Cheat Sheet the 2026-01-23.

A.1.2.4 Branches

```
## Move Between Branches
git checkout <branch>          # Switch to a different branch
  # or
git switch <branch>            # Switch to a different branch
git checkout -b <new_branch>   # Create and switch to a new branch
  # or
git switch -c <new_branch>     # Create and switch to a new branch
git branch                     # List all branches
git branch -d <branch>         # Delete a branch
git branch -D <branch>         # Force delete a branch
## Diverging branches
git switch <diverging branch>  # Switch to the diverging branch
git rebase <base branch>       # Rebase the diverging branch onto the base branch
  # or
git switch <base branch>       # Switch to the base branch
git merge <base branch>        # Merge the diverging branch into the base branch
```

A.1.2.5 Push and Pull

```
## Pushing
git push <remote> <branch>     # Push a branch to a remote repository
git push -u <remote> <branch>  # Push a new branch to a remote repository
git push --tags                # Push all tags to a remote repository
git push                       # Push the current branch to its upstream branch
git push --force-with-lease    # Force push with lease to prevent overwriting changes
##Pulling
git fetch <remote> <branch>    # Fetch a branch from a remote repository
git pull <remote> <branch>     # Fetch and merge a branch from a remote repository
git pull --rebase              # Fetch and rebase a branch from a remote repository
## Add remote
git remote add <name> <url>    # Add a new remote repository
```

A.1.2.6 Diff

```
## Diff (un)staged changes
git diff HEAD                  # Diff all staged and unstaged changes
git diff --staged              # Diff only staged changes
git diff                       # Diff only unstaged changes

## Diff commits
git show <commit>              # Show the changes introduced by a commit
git diff <commit1> <commit2>   # Show the changes between two commits
git diff <commit> <file>       # Show the changes to a file in a commit
git diff <commit> --stat       # Show the summary of changes in a commit
```

A.1.2.7 Discarding Changes

```
git restore <file>             # Discard changes to one file
  # or
git checkout <file>            # Discard changes to one file
git reset --hard               # Delete all staged and unstaged changes
git stash                      # Stash all staged and unstaged changes
git restore --staged --worktree <file> # Delete all staged and unstaged changes to a file
  # or
```

```
git checkout HEAD <file>       # Delete all staged and unstaged changes to a file
git clean                      # Remove untracked files
```

A.1.2.8 Config

```
git config user.name "Your Name"          # Set the name that will be attached to your commits
git config user.email "Your Email"        # Set the email that will be attached to your
git config --global ...                   # Set the option globally
man git-config                            # Show the manual for git-config
```

A.2 Markdown

Markdown is a lightweight markup language that allows you to format text using simple syntax. It is widely used in readme files, documentation, and blogs. Here are some basic Markdown syntax to get you started:

- Headers:

```
## Header 1
### Header 2
#### Header 3
##...
```

- Emphasis:

```
*Italic* or _Italic_
**Bold** or __Bold__
~~Strikethrough~~     # Does not work in every viewer
```

- Lists:

```
- Unordered list:
  - Item 1
  - Item 2
    - Subitem 1
    - Subitem 2

- Ordered list:
   1. Item 1
   2. Item 2
      1. Subitem 1
      2. Subitem 2

- Task list  # Does not work with every viewer
    - [x] Completed task
    - [ ] Incomplete task
```

- Links and Images:

```
[Link text](https://example.com)
[Link with title](https://example.com "Title text")
![Alt text](https://example.com/image.png)
[Reference to headers]( #header-1 )
[![Clickable Image](https://example.com/image.png)](https://example.com)
```

- Blockquotes:

```
> This is a blockquote.
- Code blocks:

```python
```
```

```
def hello_world():
    print("Hello, world!")
```

- Footnotes:

```
Here is a sentence with a footnote.[^1]

[^1]: This is the footnote text.

Or another one[^note]:

[^note]: Another footnote example.
```

- Tables:

```
|  Left  | left alt | Center | Right  |
|--------|:---------|:------:|-------:|
| Cell 1 |  Cell 2  | Cell 3 | Cell 4 |
| Cell 5 |  Cell 6  | Cell 7 | Cell 8 |
```

- Horizontal Rule:

```
---
```

- Math (using LaTeX syntax)[10]:

```
Inline math: $E = mc^2$
Block math:
$$ \int_a^b f(x) \,\mathrm{d}x $$
```

Markdown also support HTML in it. A practical way can be to change text color.

```
<span style="color:RED">...</span>
```

or use it to set precise size for image:

```
<img src="https://example.com/image.png" width="200" height="100">
```

One last tip to enhance your readme is to use badges. You can go to shields.io to generate a badge for your project. You can use them statically in your markdown to create a nice readme or documentation, or you can use them dynamically to show the status of your build, the coverage of your tests, etc.

[10] Note that not all Markdown renderers support LaTeX math. In addition, be aware to use the old delimiters `$...$` for inline math and `$$...$$` for block math, as some renderers do not support the newer `\(...\)` and `\[...\]` delimiters.